114-1 TAICA Natural Language Processing: Term Project

Checkpoint 1 and 2 of WattBot 2025

Team name: Attention Please

December 10, 2025

YouTube Video Link:
https://youtu.be/YavBekCeogE

Outline

1. Section 1: Competition Overview
2. Section 2: Dataset Specification
3. Section 3: Challenges and Problems
4. Section 4: Kaggle Competitor Solutions Review: KohakuRAG Pipeline
5. Section 5: Our Baseline Method: BM25
6. Section 6: Our HERO Pipeline (Hybrid Evidence Retrieval Optimization)
7. Section 7: Conclusion and Future Work

Section 1: Competition Overview

Context & Problem

- AI training consumes massive energy, but data is scattered in unstructured reports.
- **Gap**: Lack of transparent, evidence-based sustainability estimation.

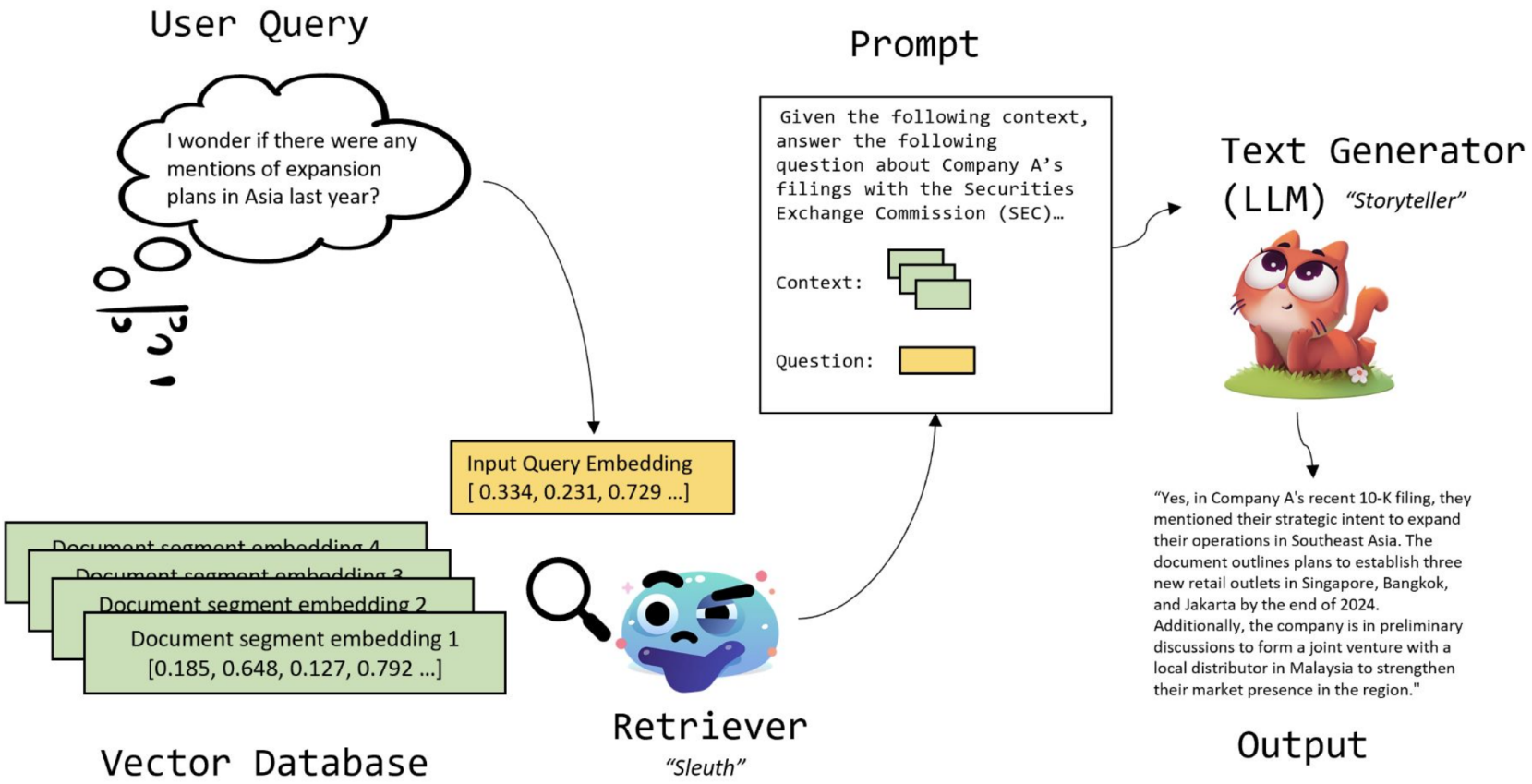
Goal: Verifiable RAG System

- Extract sustainability insights from 30+ academic papers.
- **Required Outputs**:
  - `Value` & `Unit`: Normalized numeric data (e.g., 1.2 MWh $\rightarrow$ 1200 kWh).
  - `Ref_ID`: Precise citations for traceability.

Evaluation Metrics

- **75% Accuracy**: Numeric values within $\pm 0.1\%$ tolerance.
- **15% Evidence**: Jaccard overlap of citation IDs.
- **10% Safety**: Correctly identify unanswerable queries (`is_blank`).

What is retrieval augmented generation (RAG)?



Section 2: Dataset Specification

1. **Document Corpus (30+ PDFs)**
   - **Sources**: Academic papers (arXiv) & Industry reports (Amazon).
   - **Challenge**: High heterogeneity (multi-column text, footnotes).
   - **Complex Tables**: Spanning multiple pages, merged cells, borderless.
   - **Hidden Data**: Key values embedded in charts without text descriptions.

2. **Q&A Task Types**
   - **Numeric**: Extract specific values (e.g., "CO2 emissions of BERT").
   - **Boolean**: Verify facts (e.g., "Did GPT-3 use more water than BLOOM?").
   - **Terminology**: Define specific metrics (e.g., "Carbon Intensity").
   - **Unanswerable**: Detect lack of evidence (hallucination test).

Section 3: Challenges and Problems - Data Extraction

1. **The "Table Structure" Problem**
   - **Issue**: Standard parsers treat PDFs as linear text streams.
   - **Impact**:
     - Row/Column misalignment in complex tables.
     - Values separated from headers (e.g., "1438" vs "lbs").
     - Cross-page tables broken by standard chunking.

2. **"Invisible" Visual Data**
   - **Issue**: Critical trends often exist *only* in charts.
   - **Impact**: Text-only pipelines miss this data completely (0% recall).

3. Unit Ambiguity & Normalization

- **Issue**: Mixed units (kWh vs MWh, lbs vs kg) across papers.
- **Risk**: LLMs struggle with precise arithmetic conversion.
- **Constraint**: Score implies $\pm 0.1\%$ tolerance; approximation fails.

4. Hallucination Risks

- **Issue**: RAG tends to answer even irrelevant questions (e.g., "Elephant weight").
- **Penalty**: Severe score reduction for false positives.
- **Need**: A strict "Refusal Mechanism" for unanswerable queries.

Section 4: Case Study - KohakuRAG Pipeline (Indexing)

1. **Hierarchical Indexing Structure**
   - **Tree-Based Parsing**: Documents are parsed into a semantic hierarchy:

$$\text{Doc} \rightarrow \text{Section} \rightarrow \text{Paragraph} \rightarrow \text{Sentence}$$

   - **Leaf Nodes (Sentences)**:
     - Embedded using `jina-embeddings-v3` (1024-dim).
     - Used for high-granularity vector matching.
   - **Parent Nodes (Paragraphs)**:
     - Inherit averaged vectors from children.
     - Stored alongside leaves to provide **Full Context** upon retrieval.

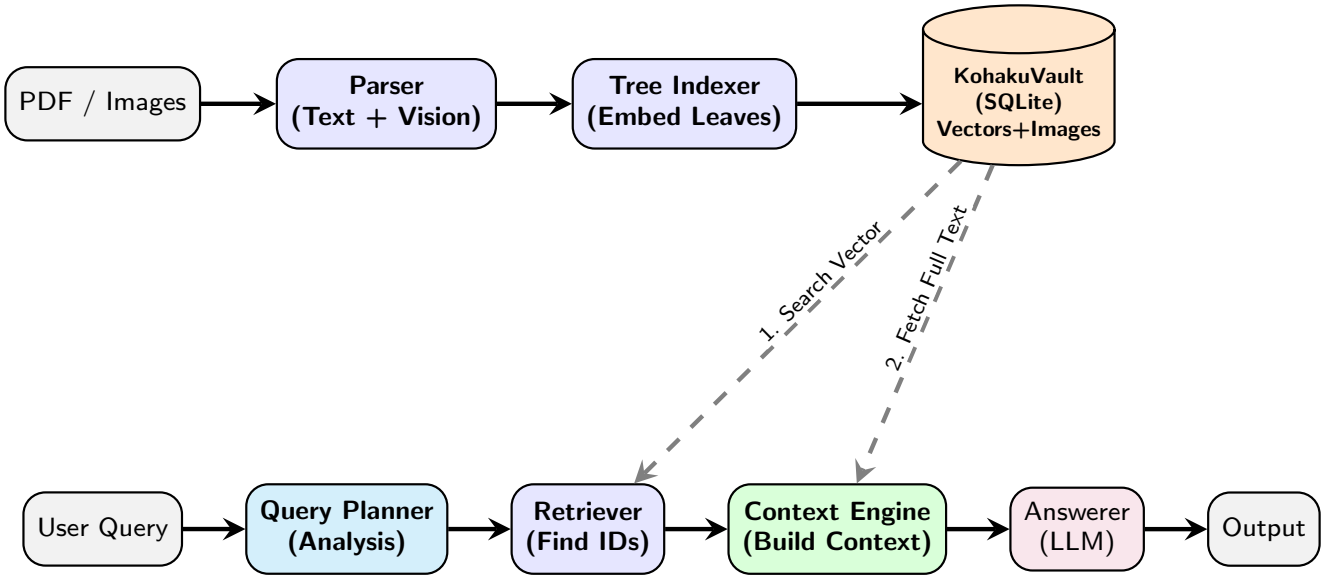
2. **Single-File Datastore (Key Differentiator)**
   - **Tech Stack**: Built on **SQLite + sqlite-vec** (via KohakuVault).
   - **Benefit**: No complex vector DB (like Faiss/Milvus) required. Entire index is a single `.db` file, ensuring reproducibility and easy deployment.

Section 4: Case Study - KohakuRAG Pipeline (Retrieval)

3. **Smart Context Expansion Strategy**
   - **Leaf-to-Root Retrieval**:
     1. **Match**: Query matches a specific **Sentence** (Leaf) via Vector Search.
     2. **Expand**: System automatically retrieves the **Parent Paragraph** and **Section Metadata**.
     3. **Result**: The LLM sees the surrounding context, not just the isolated sentence.

4. **Robust Production Features**
   - **Rate-Limit Resilience**: Implemented **Exponential Backoff** and Semaphore-based concurrency to handle OpenAI API limits automatically.
   - **Multimodal Integration**:
     - Uses Vision Models (e.g., Qwen-VL) to generate captions for figures.
     - Captions are indexed alongside text, enabling retrieval of visual data.

Section 4: Case Study - KohakuRAG Pipeline (Architecture)



Pipeline Logic:

- **Planner**: Decides *how* to search (e.g., keyword extraction).
- **Fetch Full Text**: Vector search only finds IDs. We must query the DB again to retrieve the actual paragraph text and parent context.

Section 4: Case Study - Key Innovations

1. **Tree-Based Context Expansion**
   - **Innovation**: Matches query against detailed sentences (Leaf) but automatically retrieves the full surrounding paragraph (Parent).
   - **Benefit**: Solves "Context Fragmentation." The LLM gets coherent stories, not just isolated fragments.

2. **Multimodal Layout Parsing**
   - **Technique**: Inserts tags like `[Image page=1 idx=1]` and stores generated captions (via Qwen-VL) in the same DB.
   - **Benefit**: Preserves document flow and allows retrieving visual evidence (charts/figures) alongside text.

3. **Production-Grade Engineering**
   - **Single-File Architecture**: Built on **SQLite** (KohakuVault). No external dependencies (like Pinecone/Docker), ensuring 100% reproducibility.
   - **Async I/O**: Full `asyncio` pipeline handles API rate limits (Backoff) automatically, maximizing throughput.

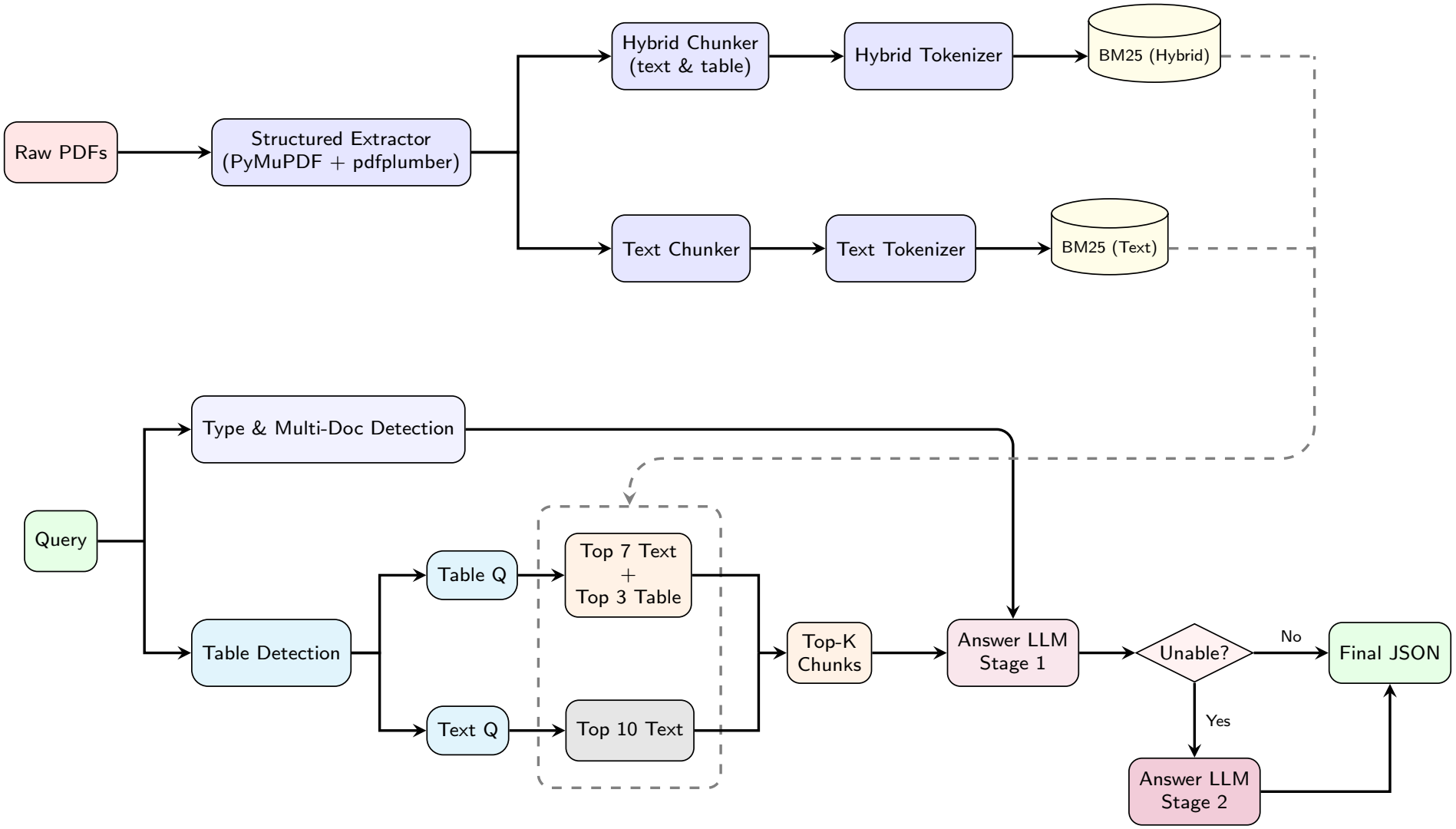
What can we learn from the case study?

1. **Structure > Chunking**
   - **Lesson**: Do not treat PDFs as flat text strings. Preserving document hierarchy (Doc $\rightarrow$ Section $\rightarrow$ Leaf) allows precise targeting.

2. **Engineering Simplicity**
   - **No Vector DB**: Using **SQLite** as a single-file datastore reduces complexity and ensures reproducibility.
   - **Resilience**: Handling API rate limits with **Backoff** logic is crucial for batch processing without crashing.

3. **Visuals are Data, Not Noise**
   - **Lesson**: Critical energy metrics are often hidden in charts. We must process images (via Vision Models/OCR), not just filter them out.

Section 5: Our Baseline Method — BM25 Sparse Retrieval



Section 5: Our Baseline Method — Data Ingestion Pipeline

1. Document Parsing Strategy

- **Tools:** PyMuPDF and pdfplumber (Standard CPU extraction).
- **Objective:** Generated two datasets to isolate the impact of "Structure Noise".

Experimental Setup: Two Dataset Versions

A. Hybrid Set (2,608 chunks)
- Text + Raw Markdown Tables

(Hypothesis: More info is better)

B. Text-Only Set (1,778 chunks)
- Pure text, tables removed

(Hypothesis: Less noise is better)

2. Enhanced Tokenization Logic Standard tokenizers split numbers, losing numeric precision. We fixed this with a custom Regex to preserve numerical entities.

```
def tokenize(text: str) -> List[str]:
text = text.lower()
# Pattern: Preserve floats, percentages, or any word character
tokens = re.findall(r'\d+\.?\d*%?|\w+', text)
return tokens
```

Impact: "H100 usage: 85.5%" → ['h100', 'usage', '85.5%'] (Value Preserved)

Section 5: Our Baseline Method — Retrieval Strategy

1. **Query Routing Mechanism**
   - Implemented a keyword-based router to detect table-dependent questions.
   - **Keywords:** `table`, `how many`, `compare`, `versus`, `GB`.

2. **Weighting Logic (The Experiment)**

Query Type	Retrieval Source	Weight
Text Query	Text-Only Index (Top 10)	1.0
Table Query	Text-Only Index (Top 7) Hybrid Index (Top 3)	1.0 **0.8**

We deliberately applied a **0.8 weight penalty** to table chunks because raw Markdown formatting (e.g., `|--- |`) acts as noise for BM25.

Section 5: Our Baseline Method — Retrieval Performance Analysis

Comprehensive Metrics

Pipeline	Recall@1	Recall@3	Recall@5	Recall@10	MRR
Hybrid (Noise)	79.49%	89.74%	92.31%	92.31%	0.8526
Text-Only (Clean)	**87.18%**	**92.31%**	**92.31%**	**94.87%**	**0.8974**

nDCG Ranking Quality

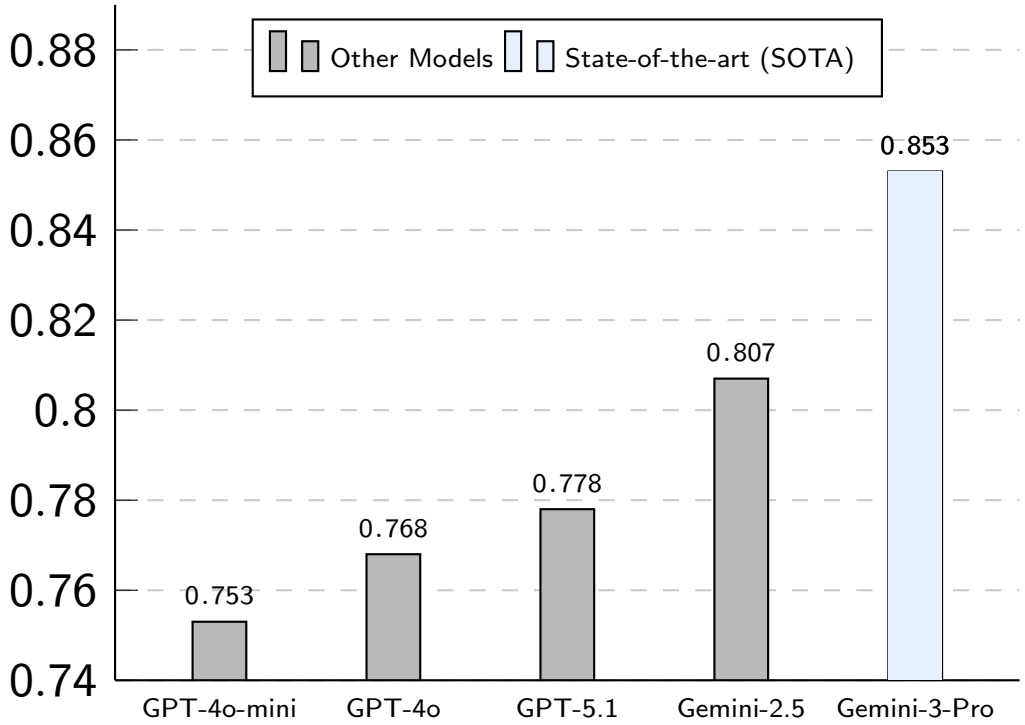
Pipeline	nDCG@1	nDCG@3	nDCG@5	nDCG@10
Hybrid (Noise)	0.7949	0.8497	0.8564	0.8617
Text-Only (Clean)	**0.8718**	**0.8859**	**0.8859**	**0.8950**

Key Insight: The "Structure Noise" Paradox

- **Text-Only Wins All Metrics:** The pipeline is superior in both coverage & ranking quality.
- **Diagnosis:** Raw table formatting acts as noise, **diluting** the Term Frequency (TF) score of actual keywords in the Hybrid index.
- **Conclusion:** BM25 cannot effectively leverage table structure, BM25 is not enough.

Section 5: Our Baseline Method — End-to-End Leaderboard

Model Performance (282 Test Queries)



Key Insight: The Retrieval Ceiling

- **LLM Power:** Gemini 3 Pro pushed the baseline score to **0.853**.
- **Bottleneck Remains:** Even with the strongest model, we are limited by the quality of BM25 retrieval (missing visual/table data).

Question: Why Focus on Retrieval?

- To break the ceiling further, we must improve **context quality** (e.g., extracting charts), not just switch LLMs.
- **Efficiency:** Baseline BM25 is cheap ($\sim$ \$0.003/query) but fundamentally noisy.

Section 5: Our Baseline Method — Limitations and Future Direction

Despite achieving **0.853**, the Baseline hits three fundamental limits:

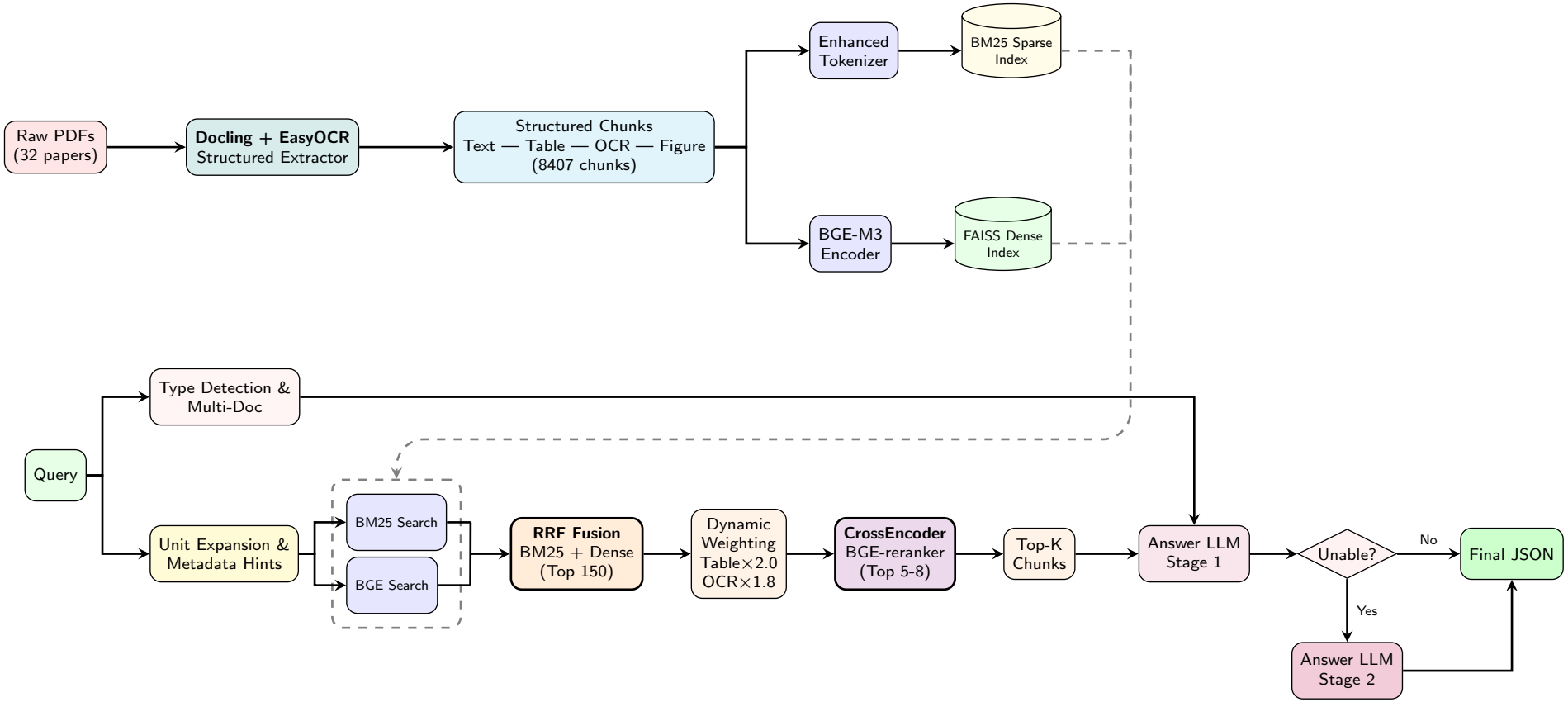
1. **Semantic Gap (Recall Issue)**
   BM25 fails on synonyms (e.g., "Carbon footprint" $\neq$ "Emission"). We miss documents that use different terminology.

2. **The Table Dilemma (Data Loss)**
   We suppressed tables (0.8 penalty) to reduce noise, but tables contain **90%** of the numerical answers. We are actively filtering out the truth.

3. **Precision Limit (Hallucination Risk)**
   Without a Reranker, Top-K results contain irrelevant chunks, which confuses the LLM and leads to hallucinations.

Solution: Introducing the HERO Pipeline!

Section 6: HERO Pipeline (Hybrid Evidence Retrieval Optimization)



Section 6: HERO Pipeline - Visual-Aware Ingestion

1. **Hybrid Parsing Pipeline (Docling + EasyOCR)**
   - **Problem**: Standard parsers ignore data embedded in charts/images.
   - **Solution**:
     - Used `Docling` to classify chunks into Text, Table, and Picture.
     - **OCR Injection**: Automatically detected figures and applied `EasyOCR` (w/ OpenCV preprocessing) to extract numerical data from charts.
     - **Result**: Converted "invisible" pixels into searchable text chunks.

2. **Enhanced Tokenization**
   - **Logic**: Custom tokenizer preserves crucial non-standard tokens:
   - **Rules**: Keeps floating point numbers, years (202x), and specific technical terms, preventing BM25 from stripping vital numeric contexts.

Section 6: HERO Pipeline - Heuristic-Weighted RRF

3. Adaptive Reciprocal Rank Fusion (RRF)

- **Concept**: Combine BM25 (Sparse) and BGE-Large (Dense) with *content-aware weights*.
- **Weighted Formula**:

$$Score(d) = \sum \frac{1}{k + rank(d)} \times \mathbf{W}_{type}$$

- **Heuristic Weights ($\mathbf{W}_{type}$)**:
  - **Tables ($\times$2.0)**: High information density for metrics.
  - **OCR/Images ($\times$1.8)**: Unique data often found only in charts.
  - **Metadata Hints ($\times$2.0)**: Matches on Author/Year in query.

4. Semantic Unit Expansion

- **Technique**: Query Augmentation via `UNIT_KEYWORDS`.
- **Example**: Query "tCO2e" $\rightarrow$ expands to "carbon emissions co2 greenhouse footprint".
- **Benefit**: Bridges the gap between abbreviated metrics and natural language text.

Section 6: HERO Pipeline - Two-Stage Adaptive Generation

5. **Dynamic Prompting Taxonomy**
   - **Classifier**: Regex-based router detects question type (e.g., `Table_Extraction`, `Score_Extraction`, `Model_Specific`).
   - **Action**: Injects specialized System Prompts (e.g., "Search row-by-row" for tables).
6. **Two-Stage Fallback Mechanism**
   - **Stage 1 (Precision)**: Strict mode using Top-3 chunks. Low temperature to prevent hallucination.
   - **Stage 2 (Recall)**: Triggered if Stage 1 returns "Unable to Answer".
     - Expands to **Top-10 chunks**.
     - Switches to **"Aggressive Mode"**: Relaxed constraints to capture partial matches or indirect evidence.

Metrics Analysis

Metric	Baseline	HERO
Recall@K (Coverage)		
Recall@1	79.49%	**80.49%**
Recall@3	89.74%	**89.80%**
Recall@5	92.31%	**92.68%**
Recall@10	92.31%	**95.12%**
nDCG@K (Ranking Quality)		
nDCG@1	0.7949	**0.8049**
nDCG@3	0.8497	**0.8502**
nDCG@5	0.8564	**0.8584**
nDCG@10	0.8617	**0.8665**
Overall Accuracy		
MRR	0.8526	**0.8560**

Key Insights:

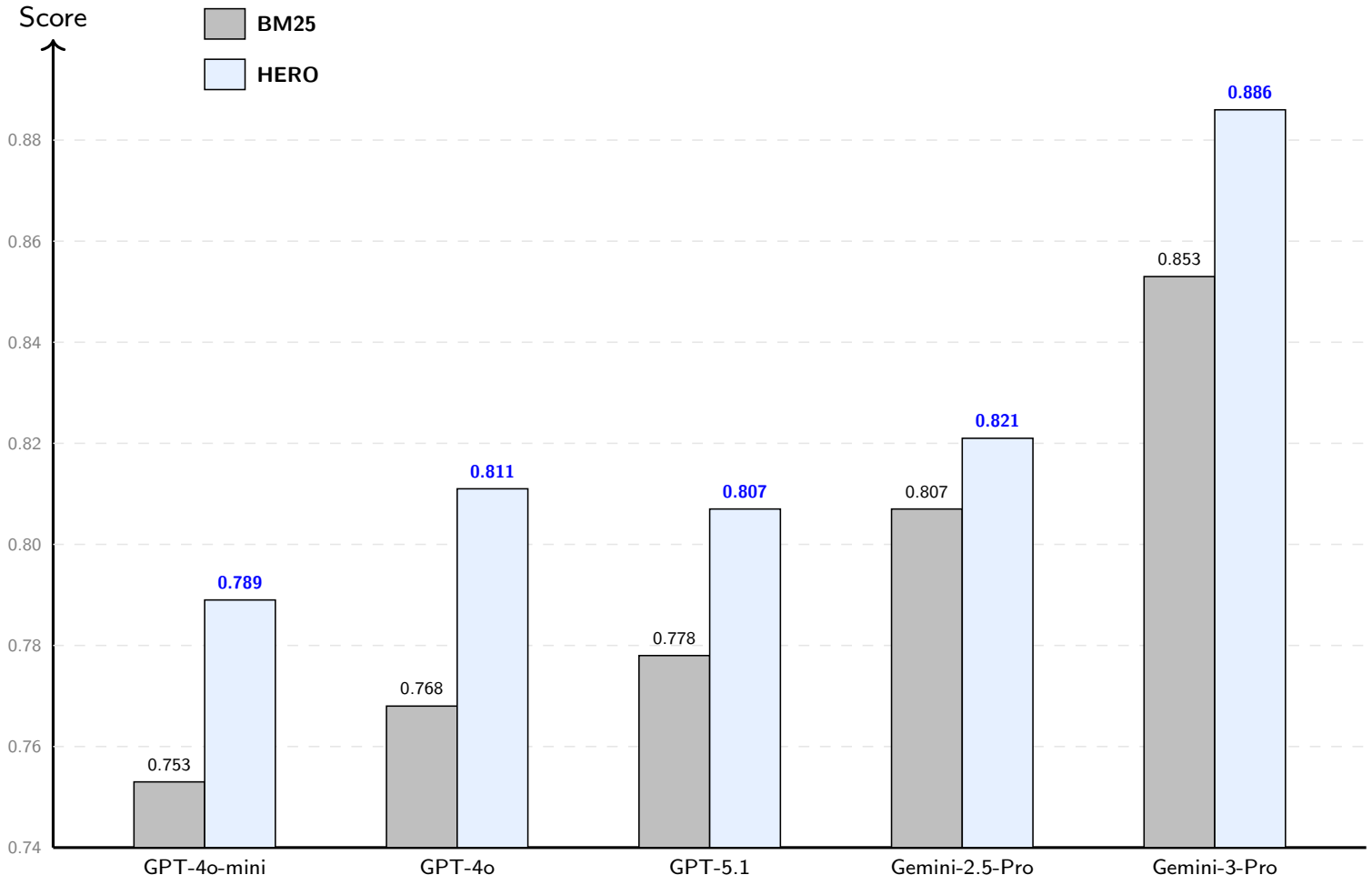
- **Precision Lift (Recall@1):** **+1%** gain. The Reranker successfully filtered the Baseline's "structure noise" and boosted ranking confidence.
- **Coverage Lift (Recall@10):** HERO reached **95.12%** coverage, proving that **Dense Retrieval** effectively found semantic matches that BM25 missed.
- **Ranking Quality (nDCG):** Overall ranking improved by **+1.7%**. This stabilization proves our Dynamic Weighting ($\times 2.0$ for Tables) works.

Key Takeaway:

HERO approach converts the Baseline's weakest point (noisy multimodal data) into its greatest strength, ensuring the LLM receives optimal context.

Section 7: Conclusion and Future Work - Pipeline & Model Benchmarking

Comparison: Baseline method vs. HERO



Section 7: Conclusion and Future Work - Key Insights

Insight 1: Unit Semantic Gap

Problem:
Vector models struggle with
`"tCO2e"` $\neq$ `"Carbon Footprint"`

Solution:
Hard-coded Query Expansion

- Map `MWh` $\rightarrow$ *Megawatt Hour*
- Map `tCO2e` $\rightarrow$ *Carbon Emission*

Semantic search alone is insufficient for technical metrics.

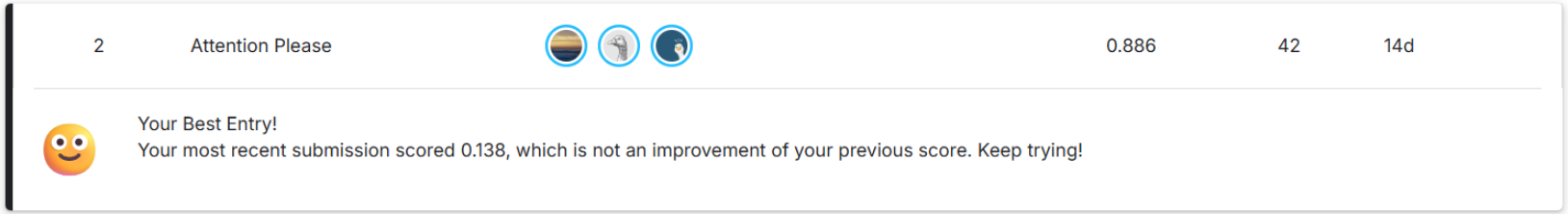
Insight 2: Safety vs. Recall

Problem:
Strict prompt $\rightarrow$ High precision
But excessive *"Unable to Answer"*

Solution:
Two-Stage Fallback

- **Stage 1:** Strict rules (Top-3, Low temp)
- **Stage 2:** Aggressive mode (Top-10, Relaxed constraints)

Balance precision with coverage.



Kaggle Leaderboard: Our HERO pipeline achieved 0.886! Top 2

Section 7: Conclusion and Future Work - Achievements

1. **Zero-to-One: Visual Data Unlocked**
   - **Challenge**: Critical energy metrics (e.g., GPU trends) are often trapped in uncopyable charts.
   - **Achievement**: Integrated `EasyOCR` into the ingestion pipeline, successfully converting pixel-based data into searchable text chunks.
   - **Impact**: Expanded information coverage from text-only to **Multimodal (Text + Vision)**.
2. **Structure-Aware Precision**
   - **Challenge**: Standard RAG treats high-density tables as normal text, diluting numeric accuracy.
   - **Achievement**: Implemented **Heuristic-Weighted RRF** (Table $\times 2.0$, OCR $\times 1.8$).
   - **Impact**: The weighted retrieval directly contributed to the **0.821** score by surfacing correct numeric answers over general descriptions.